

Hey Everyone,

Writing this message as this may be the last of my notes for many people (due to different electives).

So, first of all, THANK YOU for showing so much love to my notes over the years, I am glad that my notes could help you in exams, (& in backlogs too, maybe :)). I've not charged anyone any amount of money for my notes, they were completely free-of-cost.

But as we will now be stepping into adulthood, corporate life, etc. I have a request to make:

"If my notes ever helped you even a little bit in these 4 years of hell, consider dropping a small contribution, that would help me a lot for my Savings 😊"

As always this decision is up to you, & even if want to contribute, pls don't send ₹1 haha, it feels weird to me.



OnePlus 11 5G

By Dk | 16 May 2025 at 1:07 pm

Also, don't forget me when you become Rich & famous hehe, do save my number.

That's it from my side,
All the best for your future, may you
succeed. 😊

- D.K



Darshan Gaikwad



UPI ID: gaikwaddarshsn@oksbi

Scan to pay with any UPI app

THE LAST RIDE

★ NOTE ★

All topics covered, but still go through
Decode once

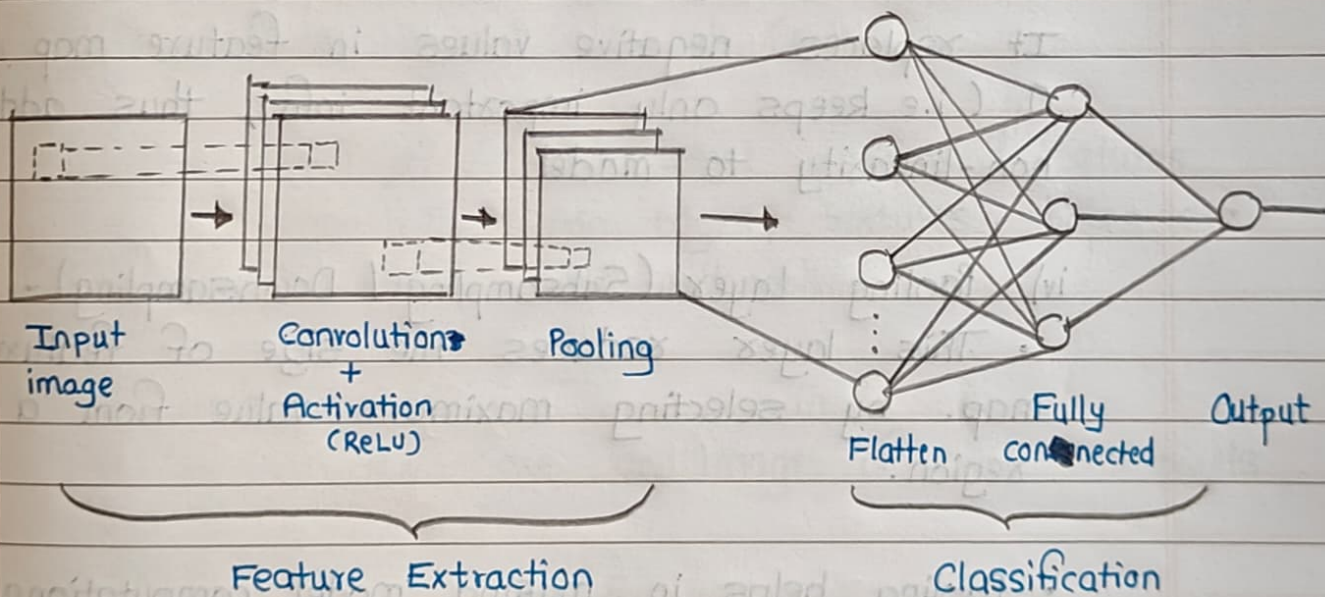
■ ■ Imp ■ ■

In my HPC Notes, I g the answer to
'One-to-All Broadcast on 8-Node Ring' is
incorrect. If incorrect, please don't refer
it.

PEACE 😊

UNIT - IIIConvolution Neural Network (CNN)★ CNN Architecture -

- • A CNN is a deep learning model specially designed to process data with grid-like structure, such as images.
- It is widely used in tasks like image classification, object detection & facial recognition.

■ Architecture -i) Input Layer -

- This layer takes an image as input. The image size is usually like $32 \times 32 \times 3$ (width $\times$ height $\times$ RGB channels).

ii) Convolution Layer -

- This is the core building block. It uses filters/kernels (eg, 3×3 or 5×5 matrix) to slide over image & detect patterns like edges or textures.

- Each filter learns to detect a special feature & multiple filters can be used in one convolution layer to extract different features.

iii) ReLU (Activation Layer) -

- ReLU stands for Rectified Linear Unit.

It replaces negative values in feature map with 0 (i.e. keeps only important info), thus adding non-linearity to model.

iv) Pooling Layer (Subsampling / Downsampling) -

- This layer reduces the size of feature map by selecting maximum value from a region.

• Pooling helps in making model computationally efficient & reduces overfitting by retaining only most important features.

v) Flatten Layer -

- This layer converts the final feature maps into a 1D vector (like regular list of numbers).

vii) Fully Connected Layer -

- Neurons in this layer are fully connected to all activations from previous layer. It combines all extracted features to make prediction.

viii) Output Layer -

- This is the final layer that gives the prediction.

★ Convolution Layer (Working) -

→ • The convolution layer is the heart of Convolutional Neural Network (CNN).

- Its main job is to detect patterns / features in an image such as edges, textures, shapes, etc as we go deeper into network.

▪ Working -

I) Suppose we have an image of size 5×5 pixels, in grayscale.

Eg,

1 1 1 0 0

1 1 1 0 0

1 1 1 0 0

0 0 0 1 1

0 0 0 1 1

II) A filter (also called kernel) is a small matrix like 3×3 .

Eg, $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$

- This filter is designed to detect vertical edges.

III) Convolution Operation -

- The filter slides over the image (left to right, top to bottom).
- At each position, we multiply corresponding values from image & filter, then add them up.
- The result is stored in a feature map (also called activation map).

iv). The feature map highlights the areas where the feature (like an edge) was found.

- The size of feature map depends on :
size of image, size of filter, stride (how many steps the filter moves) & Padding

■ Features of Convolution Layer -

i) Local Receptive Field -

- Filters focus only on small regions at a time (eg. 3×3), allowing local pattern detection.

ii) Parameter Sharing -

- The same filter is applied across the image, reducing no. of learnable parameters.

iii) Learnable filters -

- Filters are learned during training, helping model to adapt to data.

iv) Output = Feature maps :

- These maps highlight where important features are found in image.

★ Stride in CNN -

- • In CNN, stride refers to how much the filter moves across input image each time it slides.

- Normally, the filter (kernel) starts at top-left corner of image & moves across it to right & then downward.

- If the stride is 1, the filter moves one pixel at a time.

- If the stride is 2, it skips one pixel & moves two steps at a time.

- Using a larger stride is a simple & effective way to reduce spatial size (width & height) of output without using pooling.

- However there is a trade-off: larger strides can lead to loss of important details, especially in early layers where fine-grained patterns matters most.
- Hence, in many CNN architectures, initial layers often use stride = 1, while deeper layers may use larger strides to speed up processing.
- Stride is also used in other operations like pooling, where it controls how much pooling window moves.
- Eg, imagine a 5×5 input image & a 3×3 filter:
 - With stride = 1, the filter will slide 3 times across both width & height
 $\therefore$ Output size is 3×3
 - With stride = 2, it will move faster, sliding only 2 times.
 $\therefore$ Output size is 2×2 .

* Pooling Layer & Its Types -

→ • In CNN, the Pooling layer is used to reduce size (spatial dimensions) of feature maps produced by convolution layers.

- It helps in making model faster, lighter & less likely to overfit.

- Pooling works by summarizing / compressing information from small block of feature map - usually in 2×2 or 3×3 windows - and moving that window across feature map using stride.

- Need for Pooling Layer -

- i) Reduces Dimensions -

It decreases the size of feature maps, making model faster & less memory-intensive.

- ii) Prevents Overfitting -

Fewer parameters help avoid memorizing noise or irrelevant details.

- iii) Adds Spatial Invariance -

It helps model recognize features even when they are slightly shifted or rotated.

■ Types of Pooling -

i) Max Pooling -

- This is the most common type of pooling.

- It selects the maximum value from pooling window, thereby keeping most prominent feature in that region.

- Eg, from a 2×2 window like $\begin{bmatrix} 1 & 3 \\ 2 & 0 \end{bmatrix}$, it picks 3.

ii) Average Pooling -

- It calculates the average value of elements in window.

- Eg, from $\begin{bmatrix} 1 & 3 \\ 2 & 0 \end{bmatrix}$, it returns,

$$(1+3+2+0)/4 = 1.5$$

iii) Global Pooling -

- It applies pooling over the entire feature map (instead of small windows).

- It is often used just before final fully connected layer to reduce dimensions drastically.

* Features of Pooling Layer -

→ 1) Dimensionality Reduction -

- Pooling reduces the width & height of feature maps, thereby reducing memory usage & speeding up computation.

- Eg, a 4×4 feature map can become 2×2 , after 2×2 pooling with stride 2.

2) Extracts Dominant features -

- In max pooling, the most important or strongest feature in each region is selected.

3) Translation Invariance -

- Pooling provides spatial invariance - which means CNN becomes better at recognizing patterns (like edges) even if they are slightly shifted, rotated or scaled in image.

- This helps model perform better on real-world images that are not perfectly aligned.

4) Reduces Overfitting -

- Since pooling reduces the number of values passed to next layers, it lowers the no. of parameters in network.

- Fewer parameters means less chance of overfitting.

5) Improves Training Speed -

- Small feature maps after pooling means fewer computation in deeper layers.
- Hence, training becomes faster & more efficient.

6) No Learnable Parameters -

- Unlike convolution layers, pooling layers don't learn anything.
- There are no weights & biases - pooling simply performs a fixed operation (like max / average) on input data.

* Padding & Its Types -

- • In CNN, padding means adding extra pixels (usually zeroes) around border of input image or feature map.
- This is done before applying the convolution operation.
- The purpose of padding is to control the output size of feature map & to preserve edge information.
- Without padding, every time we apply a convolution, the output becomes smaller.

- Padding helps keep the spatial size consistent & ensures that features at corners & edges of image are not ignored.

■ Types of Padding -

i) Valid Padding (No padding) -

- In this, no extra pixels are added. The convolution filter only slides inside input image.

- Eg, if you have a 5×5 image & use a 3×3 filter with no padding, the output will be 3×3 .

ii) Same Padding (Zero padding) -

- In this, extra pixels (usually zeroes) are added around border of image. Padding is done in such a way that output size = input size.

- Eg, a 5×5 image with a 3×3 filter & same padding gives 5×5 output.

iii) Full Padding -

- In this, padding is added so that filter can slide even beyond original input size.

- The output size is larger than input size.

* ReLU function -

- • The ReLU (Rectified Linear Unit) is an activation function used in CNNs & other neural networks.
- It is applied after the convolution operation (or fully connected layers) to introduce non-linearity into model.
- Without activation function like ReLU, the network would behave like a linear model, no matter how many layers it has.
- The ReLU function converts all negative values to zero & keeps positive values as they are.
- This helps model focus only on strong & useful signals & ignore weaker ones.

• Formula -

$$\text{ReLU}(x) = \max(0, x)$$

so if $x > 0$, it returns x ,

& if $x < 0$, it returns 0.

- Advantages of ReLU (over Sigmoid) ← write this only if asked

i) Simplicity -

- ReLU is easier & faster to compute : just a threshold at 0.

ii) ~~ReLU~~ Avoids Vanishing Gradient Problem -

- When inputs are very high or low, the gradient becomes very small - this slows down learning.

- ReLU avoids this because it has gradient of 1 for positive inputs.

iii) Sparse Activation -

- ReLU activates only some neurons (positive ones), making model more efficient.

iv) Works well in Practice -

- Most deep learning models use ReLU because of its performance & stability.

■ Disadvantages -

i) Dying ReLU Problem -

- Sometimes, neurons output zero for all inputs & stop learning - this is called dying ReLU problem.

- This happens when weights get updated in a way that they always produce negative value, which ReLU turns into 0.

ii) Not zero-centred:

- Unlike sigmoid or tanh, ReLU outputs are always positive or 0. This can cause uneven learning in some cases.

iii) Not suitable for All Tasks -

- In tasks like binary classification, sometimes sigmoid or softmax is better for final layers.

* Dropout Layer -

→ • The Dropout layer is a regularization technique used in CNNs to prevent model from overfitting during training.

- Dropout means that during each training step, some neurons are randomly turned off (i.e. ignored).

- This means that they don't participate in forward pass (making predictions) or backward pass (updating weights).

- This helps network become more robust & prevents it from becoming too dependent on specific neurons.

■ Working -

i) During training, each neuron has a probability 'p' of being kept (eg. 0.5 or 0.8, etc).

ii) The rest $(1-p)$ of neurons are randomly set to zero for that training step.

iii) In the next training step, different neurons may be dropped.

iv) During testing (inference), dropout is turned off, but outputs are scaled to match the average.

• Dropout Layer is usually placed after fully connected layers, but sometimes they are used after convolution layers too, in complex CNN.

■ Advantages -

i) Prevents Overfitting -

• It forces the model to not rely too much on any single neuron.

ii) Simple & Efficient -

• It is easy to implement & adds no extra parameters - just randomness.

iii) Encourages Robust Feature Learning -

• Since neurons are randomly dropped, network learns multiple different paths & becomes more stable.

* Local Response Normalization (LRN) -

- • LRN is a technique used in CNNs to normalize the output of neurons, making the learning process more stable & effective.
- It was introduced in AlexNet architecture & is inspired by a concept called 'lateral inhibition' from neuroscience.
- The idea is to make most strongly activated neurons stand out more clearly, while suppressing weaker / less important ones nearby.
- This helps the network focus on most important features & ignore less useful activations.
- Need of LRN -
 - During training, - outputs of neurons can become very large / vary too much; especially after applying ReLU.
 - This can make training unstable. LRN reduces such variations & ensures that activations are balanced.
 - This helps in making learning more efficient & improving generalization (i.e. reducing overfitting).

• Working -

- LRN takes output of a neuron & divides it by a function that depends on activation of its neighboring neurons.
- This function reduces the effect of less important neurons & boosts strong signals.
- Mathematically, it uses a formula involving the square of the neighboring activations to normalize each neuron's output.

* Applications of CNN -

→ I) Image Classification -

- CNNs use convolution to extract features like corners, textures & patterns from different regions of an image.

- Eg, identifying if an image contains cat, dog, etc.

II) Object Detection -

- Object detection uses convolution not just to classify object, but also to determine whether it is in image by drawing bounding boxes.

- Eg, Detecting pedestrians, cars, etc in self-driving cars.

III) Facial Recognition -

- Convolution helps extract facial landmarks & encode them into a compact representation called embedding, which are used to identify a person.

- Eg, Unlocking phones with face ID.

IV) Medical Imaging -

- Convolution layers can detect subtle patterns in images that human eyes might miss, improving diagnostic accuracy.

- Eg, identifying tumors, fractures, etc in X-rays.

V) Satellite & Aerial Image Analysis -

- CNNs can process high-resolution satellite images to detect geographical patterns & changes over time.

- Eg, Crop monitoring, urban planning, etc.

Recurrent Neural Network (RNN)★ Unfolding Computational Graph -

- • A computational graph is a way to visually represent the series of operations done on inputs (like numbers, vectors, etc) to get an output.
- Each node in the graph represents an operation (eg, addition, multiplication, etc), & the edges represent data (values) flowing between operations.
 - Unfolding a computational graph means expanding a repeated operation (like in a loop/sequence) into a sequence of operations over time steps.
 - In simple terms, think of it like "unrolling" a loop to see what each step looks like individually.
 - This is commonly used in models like Recurrent Neural Networks (RNNs), where the same operation is repeated at each time step.

■ Example -

• Suppose we are processing a sentence, word-to-word using an RNN.

At each time step 't', RNN takes:

- The input at that time step (x_t)
- Hidden state from previous step (h_{t-1}).

- It uses a shared function 'f' to produce the new state (h_t).

Original Loop :

$$h_t = f(x_t, h_{t-1})$$

- After unfolding for 2 time steps :

Time step 1 : $h_1 = f(x_1, h_0)$

Time step 2 : $h_2 = f(x_2, h_1)$

- Each of these is a copy of same function with different inputs over time.

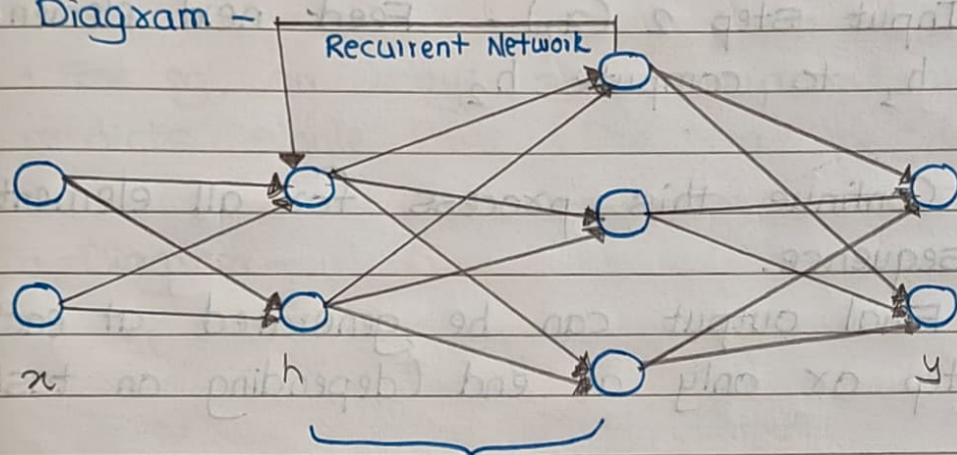
* Recurrent Neural Network (RNN) -

→ An RNN is a type of neural network that is designed to work with sequential data, such as, text, audio, time-series, etc.

- Unlike traditional neural networks (like CNN or feedforward networks), which process inputs independently, RNNs have memory of previous input.

- This means they "remember" what they've seen before & use that information to make better predictions.
- In RNNs, the outputs of previous step is fed back into network as input for next step.

■ Diagram -



Input Layer

Hidden Layers

Output Layer.

■ Working -

- RNNs process data one step at a time, passing information from one step to next using hidden state.

- At each step, RNN takes:

- the current input (x_t)
- the previous hidden state (h_{t-1})

It then produces new hidden state (h_t) & sometimes an output (y_t).

I) Start with an initial memory (h_0 , usually zeros).

II) Input step 1 (x_1) - Feed the first element of sequence into RNN

III) Use x_1 & h_0 to compute h_1 .

IV) Input step 2 (x_2) - Feed next element with h_1 to compute h_2

V) Continue this process for all elements in sequence.

Final output can be generated at each time step or only at end (depending on task).

■ Need of RNN -

- RNNs are needed for tasks that involve sequential data, where order of inputs matters.
- Traditional neural networks treat each input separately & cannot remember previous information.
- RNNs solve this by using memory (hidden state) to retain past input knowledge & use it to influence future outputs.

* Types of RNN -

→ I) One-to-One:

- This is the most basic form & doesn't actually involve sequence data.

- It refers to standard neural networks where one input gives one output.

- For eg, an image is fed into network, & it predicts single label (like "cat" or "dog").

• Diagram -



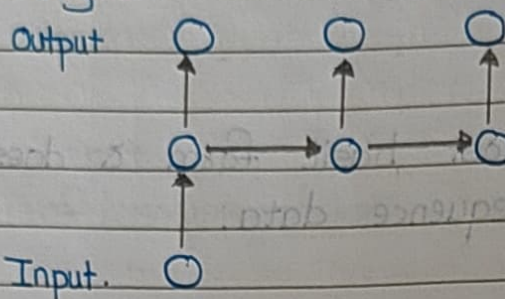
II) One-to-Many:

- In this type, a single inputs leads to a sequence of outputs.

- Eg, image captioning, where you give one image as input, & model generates a full sentence describing it.

- It uses its memory to keep track of what has been said & what should come next.

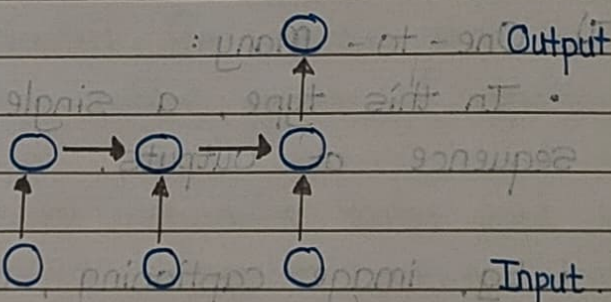
• Diagram -



III) Many-to-One:

- In this, a sequence of inputs result in a single output.
- This is commonly used in sentiment analysis, where you give a sentence (word-by-word input), & model predicts whether sentiment is positive / negative.

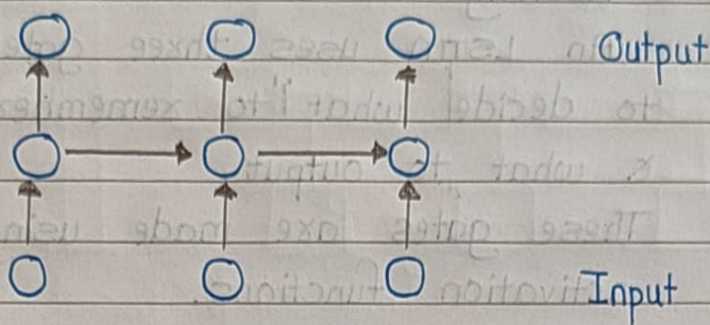
• Diagram -



IV) Many-to-Many:

- In this case, both input & output are sequences of same / different length.
- Eg, part-of-speech tagging, where each word in a sentence gets labeled with its grammatical role (like noun, verb, etc).

• Diagram -



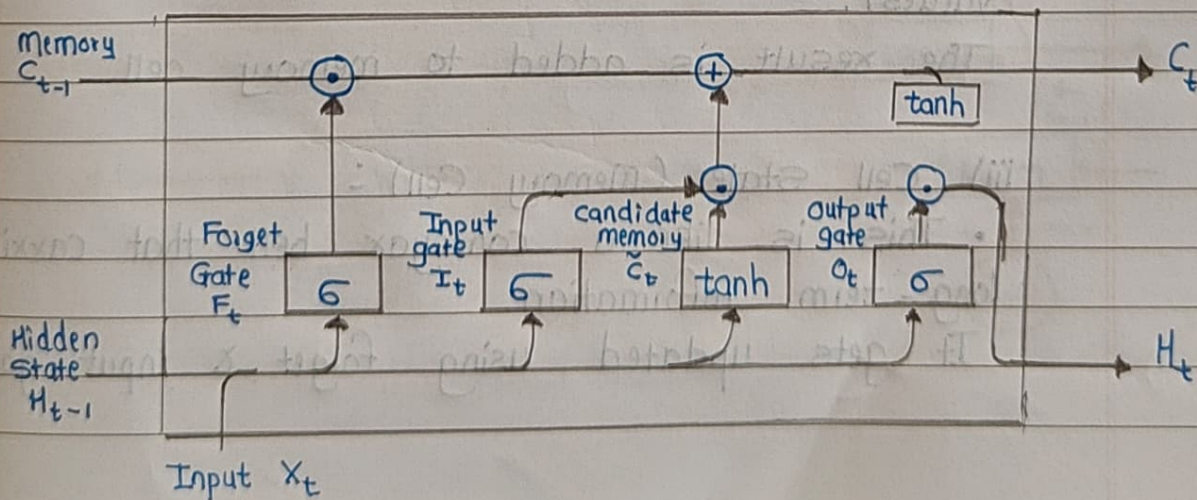
★ LSTM & Bidirectional LSTM - (Long Short-Term Memory)

→ • LSTM is a special type of RNN designed to solve problem of long-term dependencies.

• Traditional RNNs often forget earlier information when the sequence is long due to vanishing gradient problem.

• LSTMs solve this by introducing a more advanced structure with gates & memory cells that help model remember information for long periods.

■ Architecture -



■ Working -

- An LSTM uses three gates & a memory cell to decide what to remember, what to forget & what to output.

These gates are made using sigmoid & tanh activation functions.

i) Forget Gate -

- This gate decides what information from previous memory should be removed.

- It looks at current input & previous hidden state & outputs number between 0 and 1. 1 means "keep everything" & 0 means "forget everything".

ii) Input Gate -

- It controls what new information should be added to memory.

- It has two parts: a sigmoid layer (which decides which values to update) & tanh layer (that creates vector of candidate values).

The result is added to memory cell.

iii) Cell State (Memory Cell) -

- This is like a conveyor belt that carries long-term information.

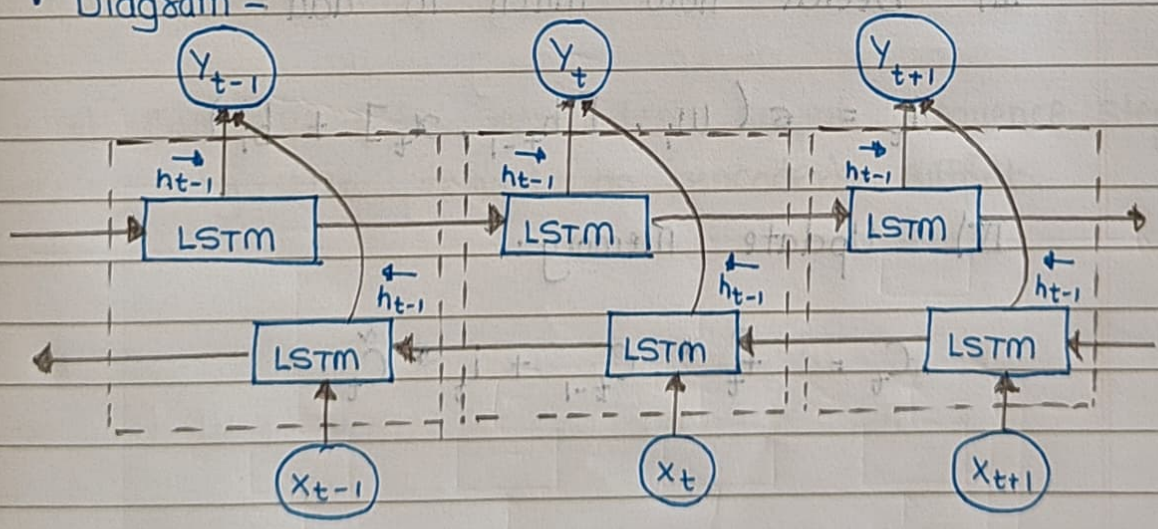
It gets updated using forget & input gates.

- iv) Output Gate -
 - It decides what part of memory should be output as hidden state.

▪ Bidirectional LSTM -

- A bidirectional LSTM is a type of LSTM that learns information in both directions - forward (past to future) & backward (future to past).
- There are two LSTM layers: one reads the sequence as it is, & the other reads it in reverse.
- The final output is combination of both directions, often by concatenation.
- It helps in tasks where context matters on both sides, like in Named Entity Recognition or Machine Translation.

• Diagram -



★ How is Memory Cell implemented computationally in LSTM -

→ • The memory cell is core of LSTM. It stores long-term information, which is updated at each time step based on input & gates.

• Internally, these computations happen:

I) Forget old memory:

$$f_t = \sigma(w_f \cdot [h_{t-1}, x_t] + b_f)$$

Multiplies with old cell state to decide what to keep.

II) Create new candidate memory:

$$\tilde{c}_t = \tanh(w_c \cdot [h_{t-1}, x_t] + b_c)$$

III) Decide how much to add:

$$i_t = \sigma(w_i \cdot [h_{t-1}, x_t] + b_i)$$

IV) Update Memory:

$$C_t = f_t * C_{t-1} + i_t * \tilde{c}_t$$

IV) Decide output:

$$o_t = \sigma(w_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

All weights ^(w) & biases ^(b) are learned during training.

★ Encoder - Decoder Sequence-to-Sequence -

→ • The Encoder-Decoder is a type of neural network used for tasks where input & output are sequences, but they may differ in length.

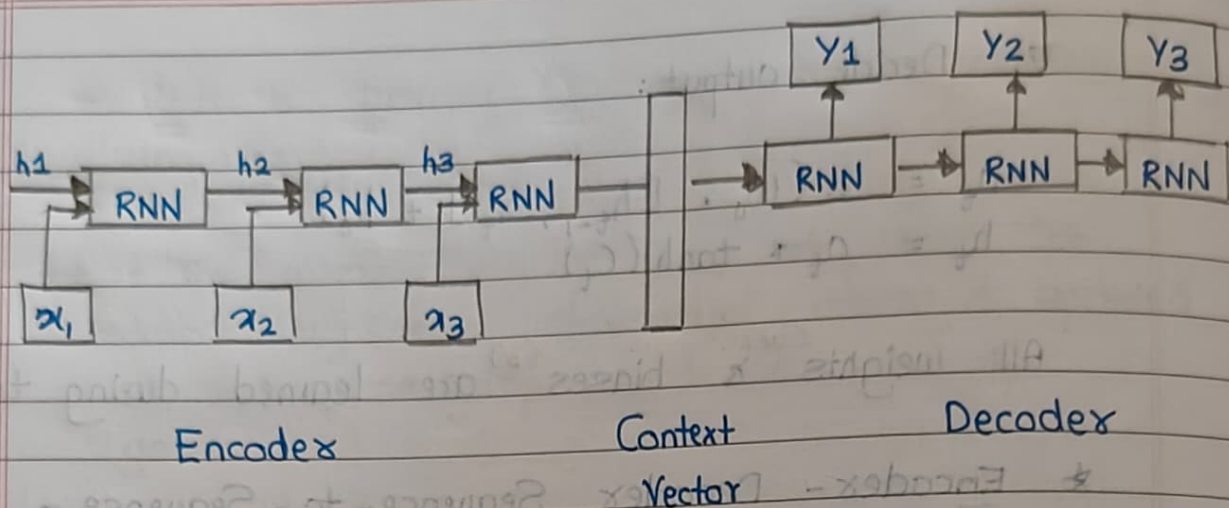
• For eg, translating an English sentence into French. It is commonly used in tasks like machine translation, text summarization, etc.

• It consists of 2 parts -

i) Encoder - It reads & understands the input sequence.

ii) Decoder - It generates output sequence step by step based on encoder's output.

■ Architecture -



i) Encoder -

- Encoder is usually a RNN, LSTM or GRU. It takes entire input sequence (like a sentence) & processes it one word at a time.

- At each step, it updates its hidden state & finally outputs a context vector, which is compressed summary of input.

ii) Context Vector -

- This is fixed-size vector generated by encoder after reading full input.

- It acts as the bridge between encoder & decoder.

iii) Decoder -

- The decoder is another RNN (or LSTM/GRU).
- It takes context vector as input, & at each time step, it predicts one output word.

- It uses its previous output & hidden state to generate the next word until it reaches the end of sentence.

* Differentiate - Recursive Neural Network vs Recurrent NN -

Recurrent Neural Network

i) RNN works on sequential data like sentences, time-series.

ii) RNN has linear chain structure, where information is passed in sequence.

iii) RNN is trained using Backpropagation Through Time (BPTT).

iv) RNN keeps a hidden state that is updated at each time step.

v) It is commonly built using LSTM / GRU.

vi) Used in Machine Translation, Speech Recognition, etc.

Recursive Neural Network

i) Recursive Neural Network works on hierarchical or structured data.

ii) Recursive network uses a tree-like structure, combining inputs in hierarchical way.

iii) Recursive NN is trained using Backpropagation Through Structure (BPTS).

iv) Recursive NN combines node representations using a fixed function at each node.

v) It is built using same neural unit recursively applied over tree structures.

vi) Used in Sentiment Analysis, Natural Language Understanding, etc.

★ Default Baseline Models -

→ • Default baseline models are simple models that we create & evaluate before trying complex models like RNN, CNN, etc.

• These models act like a starting point or reference, to compare how much better (or worse) our advanced models are performing.

• A baseline model usually involves basic rules or simple algorithms that require little or no learning.

■ Types of Baseline Models -

i) Random Classifier (for Classification) -

• This model just randomly guesses the output class.

• It sets a very low benchmark. If your trained model can't beat random guesses, it's a warning sign.

ii) Most-Frequent Class Predictor -

• This model always predicts the most common class in training dataset.

• Eg, if 80% of emails are not spam, this model always predicts "not spam".

iii) Mean or Median Predictor (for Regression) -

- This model predicts mean or median value of training data for all inputs.

- Eg, for house price prediction, if average price is 50 lakhs, it predicts 50 lakhs for every house.

iv) Rule - Based Models -

- This model uses hand-crafted rules or logic to make predictions.

- Eg, if temperature $> 30^{\circ}\text{C}$, $\rightarrow$ Predict "Hot".

- Baseline models also help to detect overfitting or unnecessary complexity in deep model.

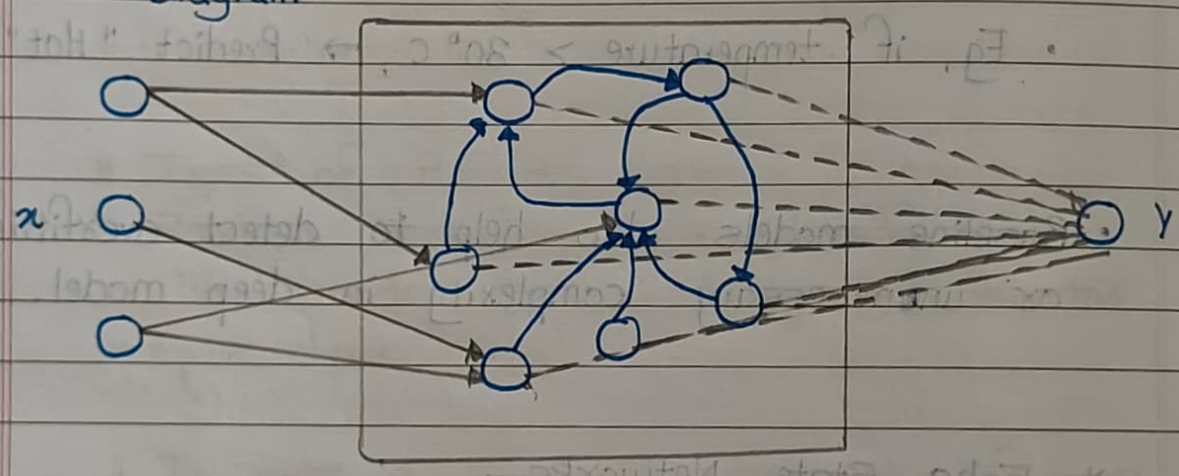
★ Echo State Networks -

- $\rightarrow$ • Echo State Networks (ESN) are special type of RNN designed to simplify training & solve long-term dependency problems in sequence learning tasks.

- Key idea of ESN is that instead of training all weights (like in traditional RNN), it only trains output layer, keeping rest of network (called Reservoir) fixed.

- The Reservoir is a large, randomly connected network of neurons.
- When input data passes through it, it creates complex patterns or "echoes" of input over time.
- These echoes help capture both short-term & long-term dependencies, which are then used to make predictions through trained output layer.

• Diagram -



Input
Layer

Reservoir

Output
Layer.

- Reservoir transforms input into high-dimensional dynamic pattern.
- This allows the network to retain memory of past inputs, even those from long ago.

- ESNs solve the vanishing gradient problem by avoiding backpropagation through time.

▪ Echo-State Property -

- This is a condition that ensures reservoir responds mostly to recent inputs, but still retains influence from older inputs.

- It balances memory & forgetting, helping the system remain stable & useful.

* Leaky Units -

- • In RNNs, Leaky Units are special kind of unit designed to help network remember information over different time scales.

- They are called 'leaky' because they slowly update their state instead of completely replacing it at each time step.

- This allows the network to retain past information for longer periods, making it easier to learn patterns that happen over both short & long durations.

- A normal RNN unit updates its memory fully with each new input, which can cause it to forget older information too quickly.

Page _____

- Leaky units solve this problem by combining a portion of old state with new state, so network can "leak" a bit of past into future.

■ Working -

- At each time step, new state of unit is weighted combination of previous state & new input.

- This helps to smooth the updates over time.

- The "Leakiness factor" (a no. between 0 & 1) controls how much of old memory is kept.

- Formula:
$$\text{new_state} = (1 - \alpha) * \text{old_state} + \alpha * \text{input}$$

where α is leak rate.

- leaky units are great at capturing patterns that occur at different time intervals because:

- i) The leakiness allows information to stay in memory longer, so the network doesn't forget important past events.

- ii) By adjusting leak rate (α), we can make some units respond quickly & others respond slowly. (for slow-changing patterns)

★ Bidirectional RNN -

→ • A bidirectional RNN is a special type of RNN that processes the input sequence in both directions: from start to end (forward) & from end to start (backward).

• It has 2 RNN layers - one moves left to right & other moves right to left & their outputs are combined at each time step.

• This design helps network to have complete context, meaning it knows what came before & what comes after each word or data point.

• For eg, consider a sentence
"He went to bank to deposit money".

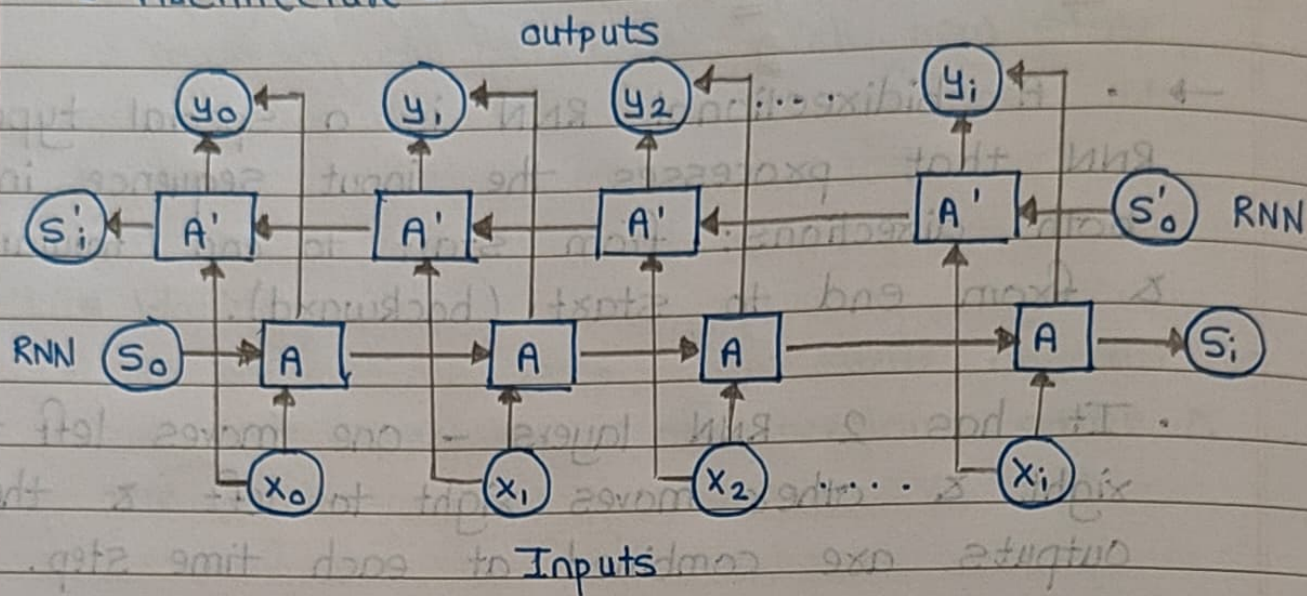
- The word "bank" can mean a riverbank or a money bank.

- If model looks only at words before "bank", it might get confused.

- But if it also sees "to deposit money", it becomes clear that it means a money bank.

• A bidirectional RNN uses both left context & right context to make better predictions.

■ Architecture -



- Input is given to both RNNs:
 - One processes input from start to end (forward RNN)
 - Another processes input from end to start (backward RNN)

■ Advantages -

- Better context understanding -
 - They use information from both past & future, improving accuracy.
- Enhanced Prediction Quality -
 - More context means more precise output, especially in NLP & time-series tasks.

* Performance Metrics of RNN -

→ 1) Accuracy -

- Accuracy is the percentage of correct predictions made by model.

- Eg, if RNN predicts 90 out of 100 words correctly, the accuracy is 90%.

2) Loss Function (Cross Entropy / MSE) -

- The loss function tells how far off the model's prediction are from actual values.

- For classification, cross entropy loss is commonly used. For regression, MSE is used.

3) Perplexity -

- Perplexity is used in language modeling tasks. It measures how well the RNN predicts a sequence of words.

- A lower perplexity value means the model is better at making predictions.

4) BLEU Score (for Translation Tasks) -

- BLEU (Bilingual Evaluation UnderStudy) score is used for machine translation tasks.

• It compares RNN-generated sentence with a reference sentence to check quality.

• The score ranges from 0 to 1 (or 0% to 100%). Higher is better.

v) Confusion Matrix -

• A confusion matrix shows actual vs predicted values in table format.

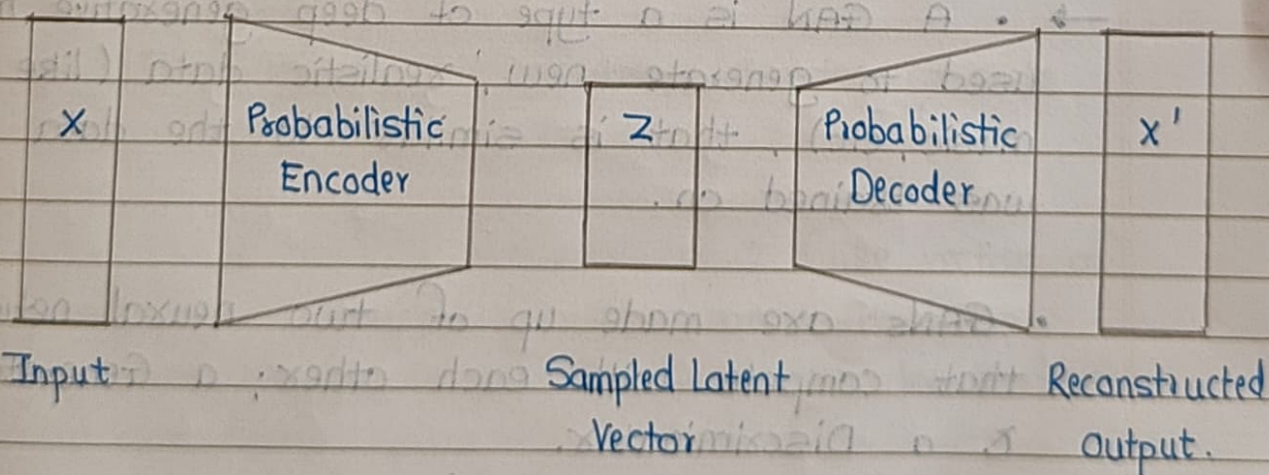
• It helps to identify where RNN is making errors - like mixing up certain classes.

• For classification, cross entropy loss is commonly used. For regression, MSE is used.

Deep Generative Models* Deep Generative Model -

- • A deep generative model is a type of deep learning model that learns to generate new data that is similar to data it was trained on.
- Unlike traditional models that only predict labels, generative models try to understand data distribution & use that to create new examples.
 - For eg, if you train a generative model on thousands of cat images, it will learn the patterns & features of cats & then be able to generate a brand-new image of cat that doesn't exist, but looks real.
 - These models are especially useful in fields like computer vision, NLP & audio generation.

• Diagram -



■ Types of Deep Generative Models -

i) Variational Autoencoders (VAEs) -

- These models compress data into smaller format (encoding) & then rebuild or generate new data (decoding).

Eg - Generate handwritten digit similar to those in MNIST

ii) Generative Adversarial Networks (GANs) -

- It is made of two networks:

a) Generator: It creates fake data.

b) Discriminator: It tries to detect if data is real or fake.

Eg - Generate realistic human faces that don't belong to real person

iii) Autoregressive Models -

- These models generate data one step at a time (i.e. predicting next value from previous ones).

Eg - Predicting & generating next word in sentence

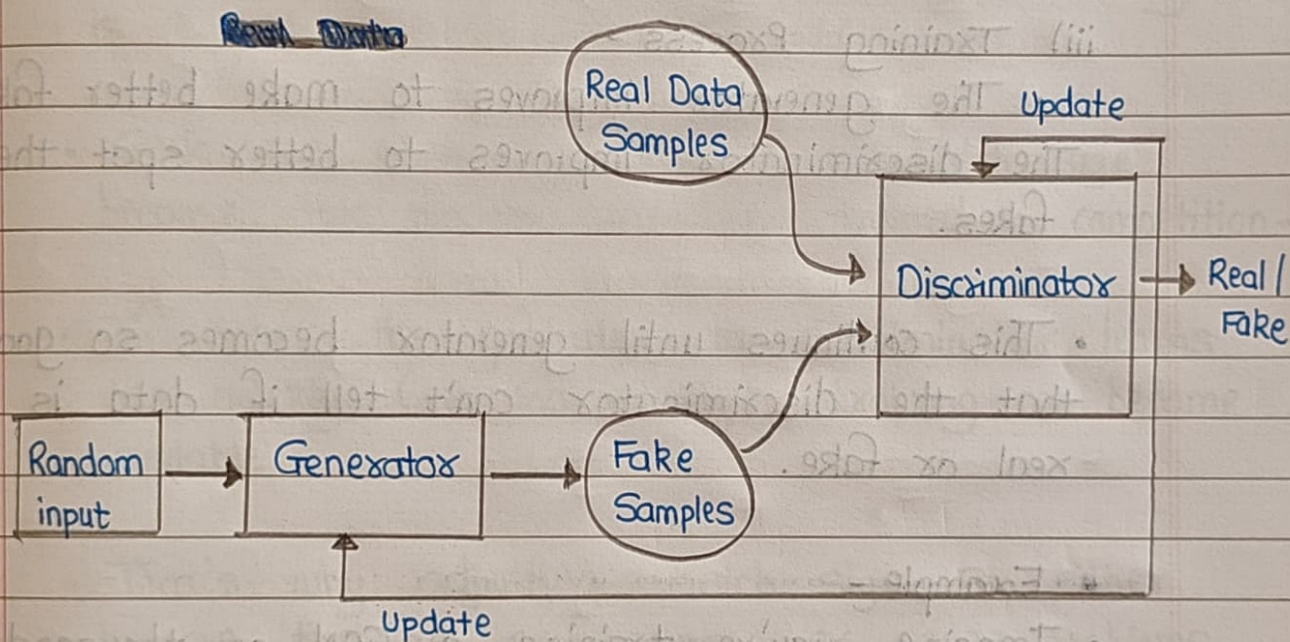
* Generative Adversarial Network (GAN) -

→ • A GAN is a type of deep generative model used to generate new, realistic data (like images, music, etc), that is similar to the data it was trained on.

- GANs are made up of two neural networks that compete with each other; a Generator & a Discriminator.

- They are trained in a game-like setup where Generator tries to fool Discriminator by creating fake data & Discriminator tries to catch it.
- Over time, this competition helps the Generator create data that looks more & more real.

■ Architecture -



i) Generator -

- The goal of generator is to generate fake data that looks real.
- It takes a random input (like vector of random numbers) & transforms it into fake image (or text / audio) that looks like real data.

iii) Discriminator -

- Its goal is to detect whether an input is real (from dataset) or fake (from the generator).

- It takes both real data & fake data as input & outputs a probability score (real or fake).

iii) Training Process -

- The generator improves to make better fakes. The discriminator improves to better spot the fakes.

- This continues until generator becomes so good that the discriminator can't tell if data is real or fake.

■ Example -

- Imagine you're training a GAN on thousands of real human face image.

- The generator starts by creating blurry or weird looking faces from random noise.

The Discriminator can easily spot these as fake

- The Generator gets feedback & improves.

- Over thousands of rounds, Generator becomes so good that it creates realistic looking faces of people who don't even exist.

Date _____
Page _____

* How does GAN training scale with batch size -

→ • In GANs, batch size refers to number of training samples passed through networks (Generator & Discriminator) in one forward-backward pass.

• The choice of batch-size has direct impact on how the model learns, how stable the training is & quality of generated results.

• GANs are especially sensitive to batch size because two models are being trained in competition.

• If either the generator or discriminator learns too quickly (or too slowly), training can become unstable.

• That's why adjusting batch size can either improve training performance or cause problems like mode collapse.

• How Batch size affects Training -

i) Small Batch Size -

• Small Batch size allows for more frequent updates, which can help the generator & discriminator stay in sync.

• However, it might slow down convergence & reduce image quality

ii) Large Batch Size -

• It produces smoother & more stable gradients, which can lead to better quality generated images.

• However, if too large, it may lead to overfitting or cause discriminator to overpower generator, resulting in training collapse.

• In general, moderate batch sizes (eg, 32, 64, or 128) work well for many GAN models.

★ Applications of GAN -

→ • GANs are widely used in field of AI because of their powerful ability to generate realistic data.

i) Image Generation & Image-to-Image Translation -

• GANs can generate new, realistic images from random noise or modify existing images.

• This is useful in many creative fields such as art, design & game development.

- GANs are also used in image-to-image translation, such as converting a sketch into real photo, a day photo into night version.

ii) Art Generation & Style Transfer -

- GANs can learn to replicate style of famous artists or create entirely new artworks by learning from various painting styles.

- Eg, a sketch can be transformed into a Van-Gogh style painting using GAN model.

- This is known as style transfer, where the content of one image & style of another are combined into new image.

iii) Medical Imaging -

- GANs help doctors by generating medical images like X-rays, MRI scans, especially in situations where real medical data is limited or sensitive.

- Eg, GANs can generate fake brain MRI scans to train a model to detect tumors more accurately.

iv) Deepfake videos & face swapping -

- GANs are used to create deepfakes - videos where person's face is swapped / altered to mimic another person's facial expression & voice.

- Eg, an actor's face can be swapped into different scenes without re-shooting using GAN.

* Types of GAN -

→ i) Vanilla GAN -

- This is the original GAN model that includes simple generator & discriminator with basic feedforward neural networks.

- Eg, generating handwritten digits (like MNIST dataset) from random noise.

ii) Conditional GAN (cGAN) -

- In cGAN, the generation process is conditioned on additional input, like class label or image.

- Both generator & discriminator receive extra information to generate more controlled & relevant outputs.

- Eg, generate images of cat only when label "cat" is given as input.

iii) Deep Convolutional GAN (DCGAN) -

- A more powerful GAN architecture that uses CNN instead of simple fully connected layers.

- DCGAN is known for generating high-quality & realistic images.

- Eg, used to generate fake celebrity faces that look extremely realistic.

iv) CycleGAN -

- CycleGAN is used for image-to-image translation without paired examples.

- It works by learning mappings between two domains using cycle consistency.

- Eg, transforming photos into paintings without having paired photo-painting data.

v) Super-Resolution GAN (SRGAN) -

- It focuses on improving resolution & quality of images - basically, it converts low-resolution (blurry) images into high-resolution (sharp) ones.

- It uses generator with residual blocks to upscale images & discriminator to ensure that generated high-res image looks realistic.

- Eg, turning 64×64 pixel blurry image to a clear 256×256 version.

vi) Wasserstein GAN (WGAN) -

- WGAN improves training stability by using different loss function based on Wasserstein distance instead of standard binary cross-entropy.

- Eg, better image generation & smooth training in cases where vanilla GAN fails.

* Deep Belief Network (DBN) -

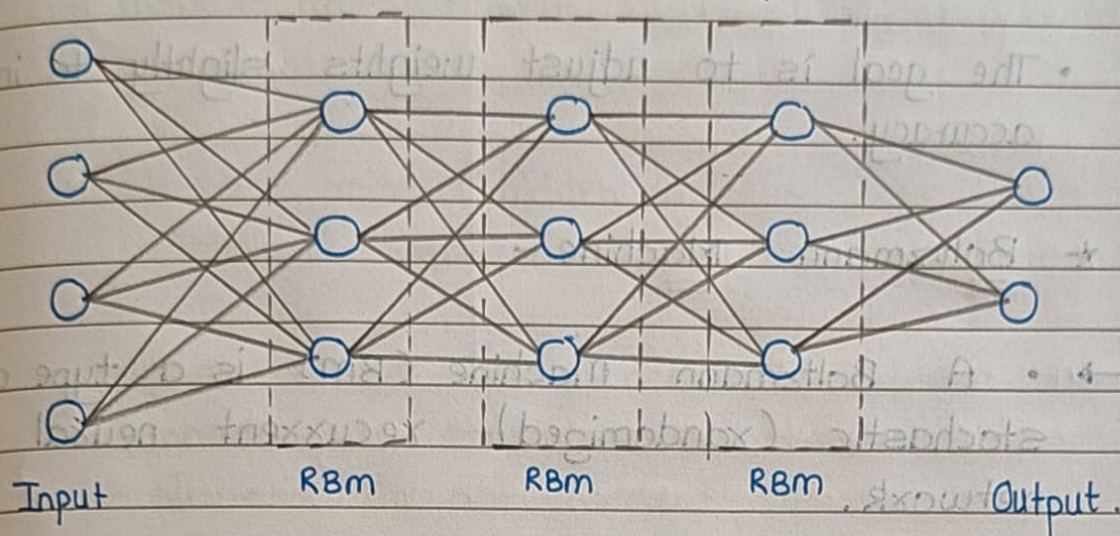
→ • A Deep Belief Network is a type of deep neural network that is built by stacking multiple Restricted Boltzmann Machines (RBMs) on top of each other.

- Each RBM learns features from previous layer.

- DBNs are unsupervised during feature learning phase & supervised during fine-tuning phase.

- They are mainly used for dimensionality reduction, feature extraction, image recognition, etc.

• Diagram - Hidden Layers



• A DBN consists of multiple layers of RBMs. Each RBM layer learns patterns & sends it to next RBM.

Working -

i) Pre-training (Unsupervised) -

• Each RBM is trained one at a time. The first RBM is trained using raw data.

• The output from first RBM becomes input for second, and so on.

• This step helps DBNs learn useful feature hierarchies.

ii) Fine-Tuning (Supervised) -

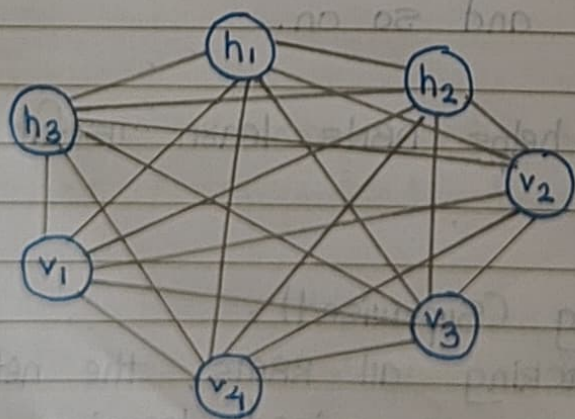
• After stacking all RBMs, the network is fine-tuned using supervised learning.

- This usually uses backpropagation & labeled data.
- The goal is to adjust weights slightly to improve accuracy.

★ Boltzmann Machine -

- • A Boltzmann Machine (BM) is a type of stochastic (randomized) recurrent neural network.
- It is used for unsupervised learning, i.e. it learns patterns & features from data without needing labeled outputs.
- Unlike feedforward networks, in a BM, all neurons are connected to each other, & the network can update its state dynamically to find optimal patterns.

• Architecture -



where, h = hidden units, &
 v = visible units

- BM consists of two types of neurons -
 - i) Visible units (input layer) :- They represent the actual data we input.
 - ii) Hidden units (hidden layer) - They learn the internal representations or patterns.
- All units are fully connected (each node connects to every other), but:
 - Neurons do not connect to themselves
 - The network is symmetrical, meaning weight from A to B is same as from B to A.
- Connections go both ways, allowing the network to reach a stable state (called energy minimization), which represents learned pattern.
- Boltzmann Machine uses a function called 'energy function' to measure how "good" a particular pattern is.
- Lower energy means better representation.

■ Objectives of BM -

- i) Learn to model the probability distribution of input data.
- ii) Discover hidden features or structures in data without supervision.

iii) Minimize energy of network to find most probable pattern.

iv) Capture dependencies between input features.

■ Types of Boltzmann Machines -

1) Standard BM -

- A Standard BM is original version where all neurons are connected to each other, i.e. every visible unit connects with other visible unit & same for hidden unit.

- This architecture is very powerful in theory because it can model complex patterns & dependencies.

2) Restricted BM (RBM) -

- A Restricted BM is simplified version of standard model, where connections within same layer are removed.

- This means visible units are only connected to hidden units & not to other visible units, & vice versa.

- This restriction significantly reduces complexity & makes the model much faster & easier to train.

3) Deep Belief Networks (DBN) -

- A Deep Belief Network is created by stacking multiple RBMs on top of each other, where output of one RBM becomes input to next.

- This structure allows network to learn multiple layers of features, starting from low-level to high-level representations.

* Differentiate - Generative Model vs Discriminative Model -

Generative Model

i) Generator's role is to create new, fake data that looks like real data.

ii) It takes random noise as input & outputs a fake data sample (image/text).

iii) It is a generative model because it creates data.

iv) It aims to fool discriminator by generating data that is indistinguishable from real data.

Discriminative Model

i) Discriminator's role is to detect whether input data is real/fake.

ii) It takes both real & fake samples as input & outputs a probability score (real/fake).

iii) It is a discriminative model because it classifies data.

iv) It aims to correctly distinguish real data from fake generated data.

v) If trained well, it can generate realistic images, audio, text that resemble real data.

v) If trained well, it becomes strong classifier that accurately detects fake data.

Extra

* Discriminator Network -

→ • In GAN, discriminator is one of the two main components.

- Its job is to act like a judge that decides whether a given input data is real (from actual dataset) or fake (generated by Generator).

- The Discriminator is binary classifier, i.e it outputs value close to 1 if data is real, & close to 0, if data is fake.

- Discriminator is usually built using deep neural networks, especially convolutional layers when working with image data.

■ Features -

i) Classifier Role -

- Discriminator is a binary classifier that labels data as either real or fake.

ii) Input Type -

- It takes two kinds of input: real data from training dataset & fake data from

generator.

iii) Loss Function -

- It uses binary cross-entropy loss to measure how well it can tell real data apart from fake data.

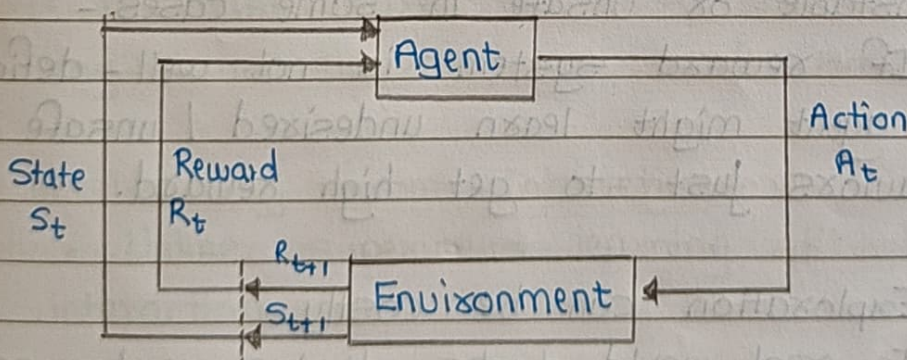
iv) Feedback Loop -

- Discriminator provides gradient signals to Generator, which helps generator improve & create more realistic outputs.

Reinforcement Learning* Reinforcement Learning -

- • Reinforcement Learning is a type of ML where an agent learns to make decisions by interacting with environment.
- The agent performs actions, & based on those actions, it receives rewards (positive / negative).
 - The goal of agent is to learn a strategy (called a policy) that maximizes total reward over time.
 - This learning process is inspired by how humans & animals learn from experience.
 - For eg, just like a child learns not to touch hot object after getting burned once, an RL agent learns what actions are good / bad by trying them & seeing the result.

■ Diagram -



■ Advantages -

i) Learns from experience -

- RL doesn't need labeled data. It learns purely by trial and error, making it ideal for problems where correct answers are not known in advance.

ii) Adapts to Dynamic environments -

- RL agents can adapt as environment changes, making them suitable for real-world, constantly evolving systems.

iii) Solves sequential decision problems -

- It is especially useful for tasks that involve sequence of decisions, such as playing games, controlling robots, etc.

■ Disadvantages -

i) Takes a lot of time & computation -

- Training an RL model can be very slow & resource-heavy, especially in complex environments.

ii) Unstable or Unsafe in some cases -

- If reward system is not well-defined, the agent might learn undesired / unsafe behaviours just to get high reward.

iii) Exploration can be costly -

- In real-world tasks, trying random actions can be risky - like testing a drone.

* Characteristics of RL -

→ i) Trial-and-Error Learning -

- RL is based on trying different actions & learning from results.
- The agent keeps experimenting & gradually improves its action based on feedback (rewards / punishments).

ii) Exploration vs Exploitation -

- A key part of RL is balancing exploration (trying new actions to discover better rewards) & Exploitation (using known actions that give high rewards).
- This balance is crucial for efficient learning.

iii) Delayed Rewards -

- The rewards in RL are often not immediate. The agent may receive actual reward after a few actions.
- It must learn to take actions that lead to long-term success, not short-term.

iv) Learning from Interaction -

- Unlike supervised learning, RL learns by interacting with environment.

- The agent perceives the current state, takes an action & receives the next state & a reward.

v) Environment - Dependent -

- The agent's learning depends heavily on structure & rules of environment.
- The same algorithm may behave differently in different environment.

vi) Policy & Value Function -

- A policy defines how agent chooses actions in different states.
- A value function estimates how good a state or action is, in terms of expected future rewards.

* Types of Reinforcement Learning -

→ 1) Positive Reinforcement Learning -

- Positive RL means rewarding the agent when it takes right action.
- This encourages the agent to repeat that action in future to earn more rewards.
- Eg, in a game, giving points to the player for making a correct move.

2) Negative Reinforcement Learning -

- Negative RL means removing something unpleasant when agent performs right action.
- This helps the agent learn to avoid bad situations & move towards better ones.
- Eg, in self-driving cars, reducing penalty or error when car stays in correct lane.

3) Model-based Reinforcement Learning -

- In model-based RL, agents build a model of environment.
- This model predicts what will happen next after an action (i.e next state & reward).
- Eg, a robot simulating different movements before actually trying them.

4) Model-free Reinforcement Learning -

- In model-free RL, agent does not build a model of environment.
- Instead, it directly learns from action & their rewards. It just remembers what worked well & uses that knowledge.
- Eg, a robot learning by trying different paths without knowing map.

5) Online Reinforcement Learning -

- In online RL, the agents learn in real-time while interacting with environment.

- It updates its strategy continuously based on new experiences.

- Eg, Chatbots improving responses as users interact.

6) Offline (Batch) Reinforcement Learning -

- In offline RL, agent learns from fixed set of past experiences stored in dataset.

- It doesn't interact with environment during training.

★ Challenges of RL -

→ 1) Exploration vs Exploitation Dilemma -

- One of biggest challenges in RL is deciding whether to explore or exploit:

- Exploration means trying out new actions to discover better rewards.

- Exploitation means using known actions that give high rewards.

- If the agent explores too much, it may waste time on poor actions.
- If it exploits too early, it may miss discovering better strategies.
- Finding right balance is crucial for learning optimal behavior.

2) Delayed Rewards -

- In RL, rewards are often not given immediately after an action.
- The agent has to perform several steps before getting feedback.
- This delay makes it hard for agent to understand which action caused reward/penalty.
- For eg, in chess, the reward (win/lose) comes only at end, not after each move, thereby making learning slow & complex.

3) Large or Continuous State Spaces -

- Many RL problems deal with environments that have huge number of possible states or continuous variables (like speed, position, etc).

- It's hard for agent to learn an optimal strategy when it cannot visit & remember every possible state.

- This leads to challenges in training, storage, etc, especially in real-world problems like self-driving cars.

4) Sample Inefficiency -

- RL often needs a large no. of training episodes (trials & errors) to learn good policy.

- This can be time-consuming & expensive.

- For eg, training a robot in real world using trial-and-error can result in wear and tear or costly mistakes.

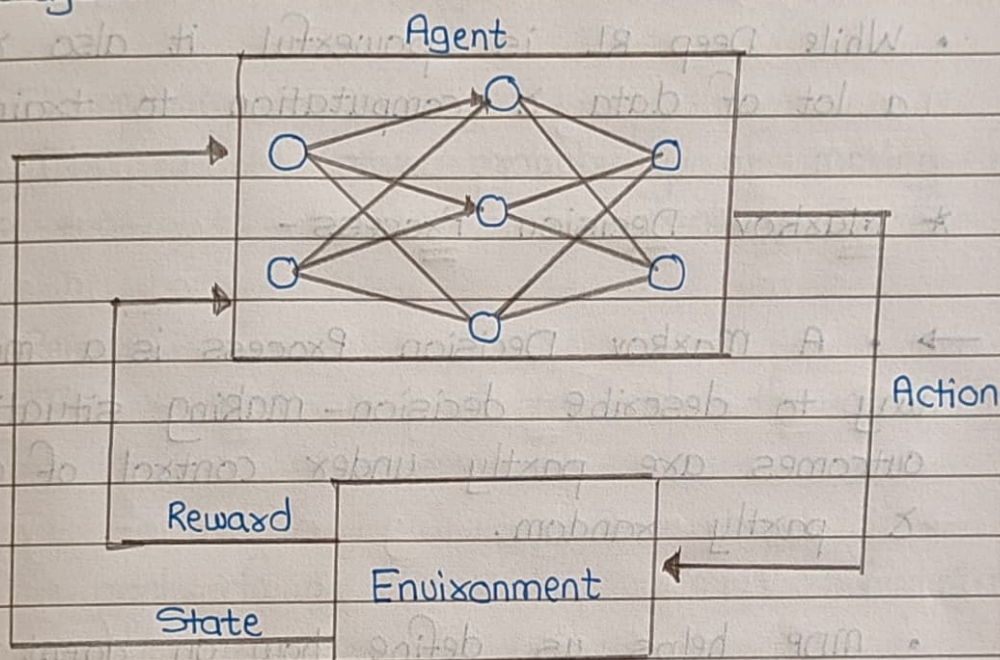
* Deep Reinforcement Learning -

→ • Deep RL is a combination of two powerful fields: Reinforcement Learning & Deep Learning.

- In Deep RL, instead of using simple tables / formulas to learn best actions, agents use Deep neural network to approximate best strategy or value of different actions.

- This allows agent to handle complex environments with high-dimensional data like images, video, etc.

• Diagram



■ Working -

- The agent observes the environment (input like an image / sensor data).
- A deep neural network processes the input & outputs the best action to take.
- The agent takes the action, receives a reward, & the neural network is updated based on that feedback.
- Over time, the agent improves its performance by learning from many such interactions.

• Example,
Self-Driving cars - they use Deep RL to learn how to navigate roads safely by interacting with simulations or real-world environments.

• While Deep RL is powerful, it also requires a lot of data & computation to train.

* Markov Decision Process -

→ • A Markov Decision Process is a mathematical way to describe decision-making situations where outcomes are partly under control of agent & partly random.

• MDP helps us define how an agent should act in each situation to maximize its long-term rewards.

• It assumes that future depends only on current state, not on past.

■ Components of MDP -

i) State (S) -

• These represent all the possible situations or configurations an agent can be in.

• Eg, in a grid-game, each cell can be state.

ii) Actions (A) -

- These are the choices the agent can make from each state.
- Eg, move left, right, up or down.

iii) Transition Probability (P) -

- This defines the probability of moving from one state to another after taking specific action.

iv) Reward (R) -

- The reward is the feedback, the agent gets after taking an action.

- The goal of agent is to maximize the total reward over time.

v) Policy (π) -

- A policy is a strategy that the agent follows - it maps states to actions.

■ Markov Property -

- The Markov property states that "The next state depends only on current state and action, not on sequence of events that came before it."

- Example, imagine a robot moving in a room:

a) State - Current location of robot

b) Actions - Move forward, turn left, turn right.

c) Transition - If robot moves forward, it may land in a new spot with some probability

d) Rewards - If robot reaches the charging station, it gets +10 reward.

e) Markov Property -

The robot only needs to know where it is now to decide what to do next - it doesn't need to remember where it was 10 steps ago.

* Dynamic Programming in RL -

→ • Dynamic Programming is set of algorithms used to solve RL problems when we have complete & accurate model of environment (i.e. when we know states, actions, rewards & transition).

• These methods help us compute optimal policy (best way to act) & Value functions (how good it is to be in a state).

• DP solves problems by breaking them into subproblems, solving smaller parts & combining their results.

■ Various DP Algorithms in RL -

i) Policy Evaluation -

• This algorithm computes value function $V(s)$ of given policy (π).

- It uses Bellman Expectation Equation repeatedly to update value of each state until the values converge.

- It is mainly used to find out how good a fixed policy is before improving it.

ii) Policy Improvement -

- Once you have value function for a policy, you can improve the policy by acting greedily - i.e. choosing actions that lead to best values.

- This step updates the policy to choose actions that lead to higher rewards.

iii) Policy Iteration -

- This combines 'Policy Evaluation' & 'Policy Improvement' in a loop:

a) Evaluate the current policy

b) Improve the policy based on updated values.

c) Repeat until the policy stops changing.

- This results in finding the optimal policy.

iv) Value Iteration -

- Instead of separately evaluating & improving policy, this method updates value function directly using Bellman Optimality Eqⁿ.

- It's more efficient & faster than policy iteration in some cases.

- Once the value function converges, the optimal policy can be derived from it.

★ Q - Learning & Deep Q - Networks -

→ ■ Q - Learning -

- Q - Learning is a type of model-free RL algorithm.

- It helps an agent to learn how to act optimally in an environment by learning quality (Q-value) of taking an action in particular state, without knowing full environment in advance.

- It works by updating Q-values using Bellman equation.

- The Q-value tells the agent how good it is to take a specific action from given state, aiming to maximize total reward over time.

■ Features -

i) Q - table - based :

- It uses a table to store Q-values for each state-action pair $Q(s, a)$.

ii) Learning from experience:

- The agent explores the environment & updates Q-values using rule:

$$Q(s,a) = Q(s,a) + \alpha [R + \gamma * \max_{a'} (Q(s',a')) - Q(s,a)]$$

where, α = learning rate

γ = discount factor

R = reward received

$\max(Q(s',a'))$ = max future reward from next state.

iii) Exploitation vs Exploitation -

- The agent uses ϵ -greedy strategy: It sometimes explores new actions (exploration) & sometimes uses best known action (exploitation).

■ Deep Q-Network -

- Q-learning doesn't work well for large or continuous state spaces because it tries to store all Q-values in a table.

- To overcome these limitations of traditional Q-learning, Deep Q-Network was introduced.

- It uses a neural network instead of Q-table to approximate Q-values, allowing it to handle large / continuous state spaces.

■ Features -

i) The neural network takes state as input & outputs Q-values for all possible actions.

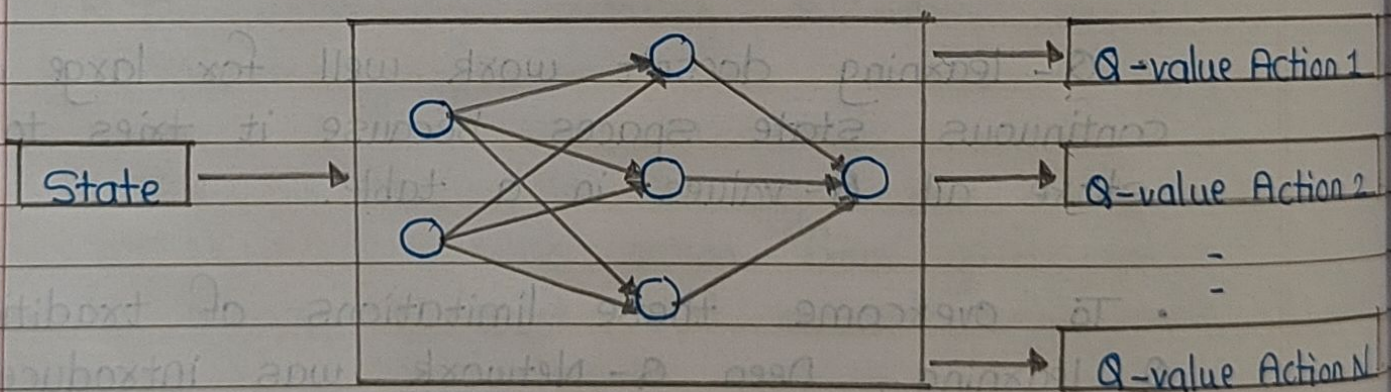
ii) DQN stores past experiences (state, action, reward, next state) in a memory buffer.

• It trains network on random batch of these experiences to reduce correlation & improve learning.

iii) DQN uses separate target network to make learning stable.

• This network is updated less frequently & helps reduce oscillations during learning.

■ Diagram -



* Simple Reinforcement for Tic-Tac-Toe -

→ 1) State Representation -

- The current board is considered a "state".
- There are many possible board states. The state can be stored as a string or a tuple showing current layout of "X", "O" & empty spots.

2) Action Space -

- The actions are the valid moves (empty spots where symbol can be placed.)
- At each state, the agent chooses one of available actions.

3) Learning through Self-Play -

- The agent plays thousands of games, often against itself.

- It tries different moves, remembers which leads to win or loss, & learns from these outcomes.

4) Reward System -

- +1 reward for winning the game
- 0 reward for a draw
- -1 reward for losing

- 5) Policy update (Value function / Q-values) - *
- The agent stores a value for each state that reflects expected future reward.
 - These values are updated after each game based on outcome.

- 6) Exploitation vs. Exploration -
- Sometimes, the agent tries random moves (exploration) to learn new strategies.
 - Other times, it uses best known move (exploitation).

* Compare - Active vs Passive RL -

Active RL

i) The agent actively decides which actions to take.

ii) It learns both, the optimal policy & value function

iii) It has full control over its actions

Passive RL

i) The agent follows a fixed policy & doesn't choose actions freely.

ii) It learns only the value of fixed policy.

iii) It has no control, just observes & learns from given actions.

iv) Its goal is to find best possible strategy by trial & error

v) Used when agent needs to learn best way to act.

vi) Eg, robot deciding best way to navigate maze.

iv) Its goal is to evaluate how good the fixed strategy is.

v) Used when agent already has strategy & wants to evaluate it.

vi) Eg, robot following set path & learning how good it is.